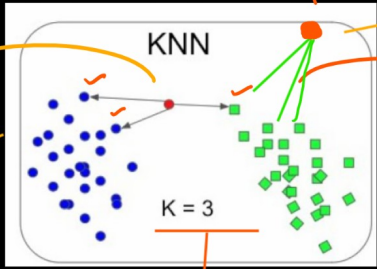


Anomaly Detection → Unsupervised ML
 → KNN → classification
Regression



$$\left. \begin{array}{l} \text{1st neighbor} = 0.9 \\ \text{2nd neighbor} = 1 \\ \text{3rd neighbor} = 1.2 \end{array} \right\} \Rightarrow \frac{0.9 + 1 + 1.2}{3} \Rightarrow \underline{\underline{1.03}}$$

neighbors (hyperparameters)

$$\left. \begin{array}{l} \text{1st neighbor} = 4 \\ \text{2nd neighbor} = 6 \\ \text{3rd neighbor} = 8 \end{array} \right\} \Rightarrow \frac{4 + 6 + 8}{3} \Rightarrow \underline{\underline{6}}$$

```
from sklearn.neighbors import NearestNeighbors
nbrs = NearestNeighbors(n_neighbors=5).fit(X)
distances, indices = nbrs.kneighbors(X)
knn_scores = distances[:, -1] # distance to the 5th nearest neighbor
```

```
threshold = np.percentile(knn_scores, 95)
anomalies = X[knn_scores > threshold]
```

```
plt.scatter(X[:, 0], X[:, 1], color = 'blue', s=20, label='Inliers')
plt.scatter(anomalies[:, 0], anomalies[:, 1], color='red', s=30, label='Anomalies')
plt.legend()
plt.show()
```

2 things → ① for every data point, I want to know its 5 nearest neighbors.

$\begin{bmatrix} [0.2, 0.4, 0.5, 0.7, 0.9], \\ [0.1, 0.2, 0.3, 0.5, 0.6], \\ [0.5, 0.8, 1.2, 1.5, 4.0] \end{bmatrix}$

② It simply stores the dataset internally

$X = \begin{bmatrix} [2, 3], \\ [5, 4], \\ [7, 8] \end{bmatrix}$

→ Remember these coordinates

0	—	A	→
1	—	B	0.2
2	—	C	0.3
3	—	D	0.6
4	—	E	0.8
5	—	F	1.2

KNN → solely relies on distance
 (Huge amount of data) evaluation

very huge amount of data (nonlinearity)
 ↳ SVM

They can be effective for anomaly detection because they find the hyperplane that best separates the normal data points from the anomalies.

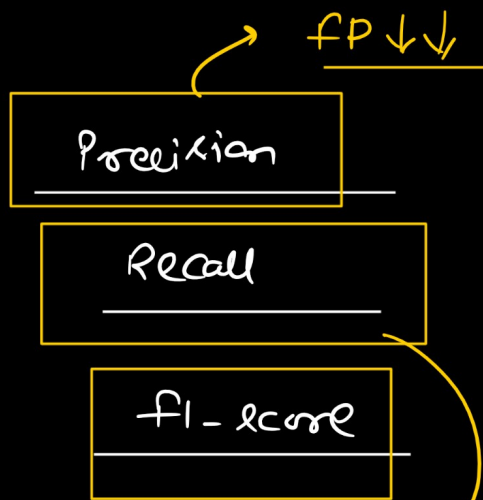
One class SVM

↳ unsupervised technique

(Normal → 1
 Anomalies → -1)

Confusion Matrix

↓
fraud detection system



100 transactional records

95 are genuine

transactional

5 are false
alarms

↘ tree data structure

Task → Isolation Algorithm

Jan → 1 lakh	10,000
Feb → 0	20,000
Mar → 10 lakh	15,000
April → 50 lakh	25,000
June → 500	20,000
July → ?	28,000